

Intro to CTF & Kali Linux - Challenges Writeup

1. Super SSH

Category: General Skills

Challenge Description

The challenge requires us to connect to a remote server using Secure Shell (SSH) with the provided username, hostname, port, and password to retrieve the flag.

Solution

Step 1: Connect to the SSH server

Use the `ssh` command with the provided username, hostname, and port number.

```
ssh ctf-player@chatelaine.cylabacademy.net -p 18783
```

- `ssh` — Starts a Secure Shell connection.
- `ctf-player` — The username used to log in.
- `chatelaine.cylabacademy.net` — The remote server's hostname.
- `p 18783` — Specifies the SSH server's port number.

Step 2: Accept the host fingerprint

On the first connection, SSH asks whether you trust the server's host key.

```
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

Type `yes` and press Enter to accept the fingerprint, as instructed by the challenge.

Step 3: Enter the password

When prompted, enter the provided password:

```
fd7746b4
```

The password will not appear on the screen while you type. This is normal behavior for SSH password prompts.

Step 4: Retrieve the flag

After successful authentication, the server displays the flag:

```
Welcome ctf-player, here's your flag: academy{s3cur3_c0nn3ct10n_7a8547e8}
```

Flag

```
academy{s3cur3_c0nn3ct10n_7a8547e8}
```

Key Takeaways

- SSH allows users to securely access remote systems over a network.
- The `-p` option specifies a custom SSH port when the server does not use the default port `22`.
- SSH uses host keys to help clients verify the identity of the server.
- Authentication requires valid credentials, such as a username and password.

Conclusion

This challenge demonstrates how to establish an SSH connection to a remote server using a custom port and password. Once authenticated successfully, the flag is provided directly by the server.

2. WebDecode

Category: Web Exploitation

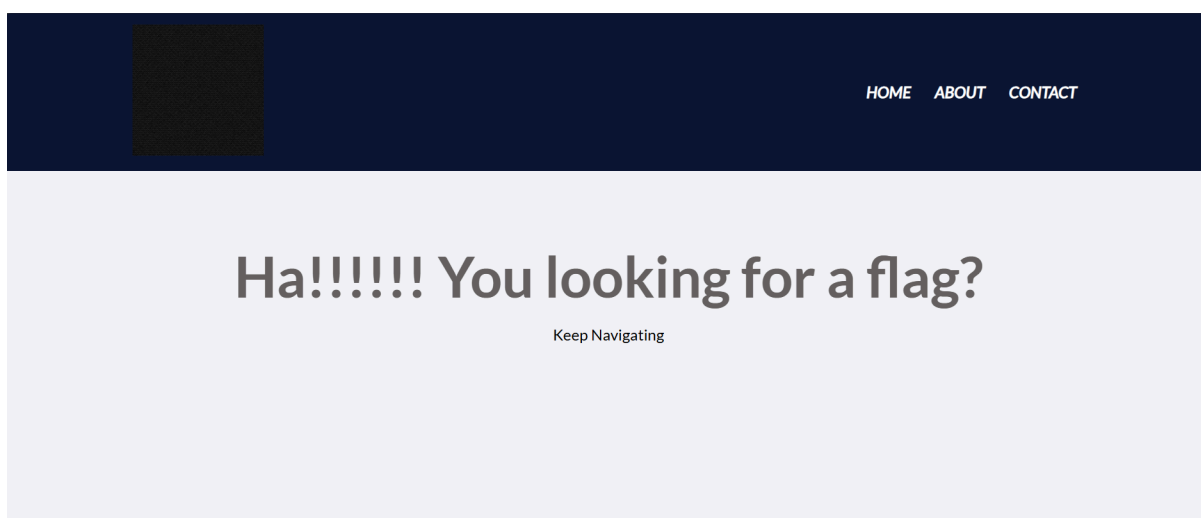
Challenge Description

The challenge introduces the use of a web inspector to investigate a webpage and find a hidden flag. The flag may be stored in the page's HTML or encoded in a format that needs to be decoded.

Solution

Step 1: Inspect the webpage

Open the challenge website and observe the message displayed on the page:



The message suggests that we need to investigate the website further.

Step 2: View the page source

Right-click the webpage and select **View Page Source**, or press `Ctrl + U`.

The HTML source reveals links to other pages, including `about.html` and `contact.html`.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initia
```

```

l-scale=1.0">
  <link rel="stylesheet" href="style.css">
  <link rel="shortcut icon" href="img/favicon.png" type="image/x-icon">
  <!-- font (google) -->
  <link href="https://fonts.googleapis.com/css2?family=Lat
o:ital,wght@0,400;0,700;1,400&display=swap" rel="styleshee
t">
  <title>Home</title>
</head>
<body>
<header>
  <nav>
    <div class="logo-container">
      <a href="index.html"></a>
    </div>
    <div class="navigation-container">
      <ul>
        <li><a href="index.html">Home</a></li>
        <li><a href="about.html">About</a></li>
        <li><a href="contact.html">Contact</a></li>
      </ul>
    </div>
  </nav>
</header>
  <section class="banner">
    <h1>Ha!!!!!! You looking for a flag?</h1>
    <p>Keep Navigating</p>

  </section><!-- .banner -->
  <section class="sec-intro">
    <div class="col">
      <p>Haaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa</p>
      <p>Keppppppppppppppppppp Searchinggggggggggggggggggggg</p>
      
      <figcaption>Don't give up!</figcaption>

```

```
</div>
</section><!-- .sec-intro -->

<footer>
  <div class="bottombar">Copyright © 2023 Your_Name. All
rights reserved.</div>
</footer>

</body>
</html>
```

Since the homepage does not contain the flag, navigate to `about.html` and inspect its source code.

Step 3: Find the encoded flag

In the HTML source of `about.html`, look for the following line:

```
<section class="about" notify_true="YWNhZGVteXt3ZWJfc3VjYzNzc2Z1bGx5X2QzYzBkZWRFMDc5ODliMjV9">
```

The `notify_true` attribute contains a suspicious string that looks like Base64 encoding.

Encoded data can be stored in HTML attributes, comments, scripts, or other parts of a webpage. This is why inspecting the source code is useful during web exploitation challenges.

Step 4: Decode the string using CyberChef

1. Copy the encoded string from the `notify_true` attribute:

```
YWNhZGVteXt3ZWJfc3VjYzNzc2Z1bGx5X2QzYzBkZWRFMDc5ODliMjV9
```

2. Open CyberChef.
3. Paste the string into the **Input** field.
4. Search for the **From Base64** operation and add it to the recipe.
5. Read the decoded result in the **Output** field.

The output reveals the flag.

The screenshot shows the CyberChef web application interface. On the left, the 'Recipe' panel is active, showing a 'From Base64' operation. The 'Alphabet' dropdown is set to 'A-Za-z0-9+/=' and the 'Remove non-alphabet chars' checkbox is checked. Below the recipe panel, there is a 'STEP' button and a green 'BAKE!' button with a chef icon. On the right, the 'Input' panel contains a long Base64-encoded string: 'YWNhZGVteXt3ZWJfc3VjYzNzc2Z1bGx5X2QzYzBkZWRFMDc5ODliMjV9'. The 'Output' panel shows the decoded result: 'academy{web_succ3ssfully_d3c0ded_07989b25}'. The interface also includes a status bar at the bottom with 'nbc 42' and '1'.

Flag

```
academy{web_succ3ssfully_d3c0ded_07989b25}
```

Key Takeaways

- **View Page Source:** Allows us to inspect a webpage's HTML code.
- **HTML attributes:** Hidden information can be stored in custom attributes, even if it is not visible on the webpage.
- **Base64 decoding:** CyberChef can decode Base64-encoded strings using the `From Base64` operation.
- **Web inspection:** Always inspect linked pages and their source code when searching for hidden information in web exploitation challenges.

Conclusion

The flag was hidden in a custom HTML attribute on the `about.html` page and encoded using Base64. By inspecting the source code and decoding the suspicious string with CyberChef, we successfully retrieved the flag.

3. information

Category: Forensics

Challenge Description

Files can always be changed in a secret way. Can you find the flag?

The challenge provides an image file named `cat.jpg`. Our goal is to investigate the file and discover hidden information that reveals the flag.



Solution

Step 1: Analyze the image using Aperi'Solve

Instead of using `exiftool` locally, we can use Aperi'Solve to analyze the image online.

1. Open the Aperi'Solve website.
2. Upload the provided `cat.jpg` file.
3. Wait for the analysis to finish.
4. Examine the results, especially the image's metadata.

Aperi'Solve runs several image analysis tools and displays their results in one place, making it useful for beginner-friendly forensics challenges.

Step 2: Find the suspicious metadata

In the metadata results, look for the **License** field. It contains the following Base64-encoded string:

```
YWNhZGVteXt0aGVfbTN0YWRhdGFfMXNfbW9kawZpZW9
```

The string looks like Base64, so we need to decode it to reveal the hidden information.

Step 3: Decode the string using CyberChef

1. Open [CyberChef](#).
2. Copy the encoded string from the License field.
3. Paste it into the **Input** field.
4. Add the **From Base64** operation to the recipe.
5. Read the decoded text in the **Output** field.

The decoded output reveals the flag.

The screenshot shows the CyberChef web interface. On the left, the 'Recipe' panel is active, displaying a 'From Base64' operation. The 'Alphabet' dropdown is set to 'A-Za-z0-9+/' and the 'Remove non-alphabet chars' checkbox is checked. At the bottom of the interface, there is a green 'BAKE!' button with a chef icon and an 'Auto Bake' checkbox. On the right, the 'Input' panel contains the Base64 string 'YWNhZGVteXt0aGVfbTN0YWRhdGFfMXNfbW9kawZpZW9'. Below the input, a status bar indicates '44 selected'. The 'Output' panel shows the decoded result: 'academy{the_metadata_is_modified}'.

Flag

```
picoCTF{the_m3tadata_ls_modified}
```

Key Takeaways

- **Metadata:** Files can contain additional information beyond their visible content, such as copyright details, timestamps, and license information.
- **Aperi'Solve:** An online tool that automates multiple image analysis techniques and displays the results.
- **Base64 decoding:** Encoded strings found in metadata may contain hidden messages or flags.
- **Forensics methodology:** Do not focus only on the image itself. Always examine its metadata for suspicious or unexpected values.

Conclusion

The flag was hidden in the image's metadata, specifically in the License field, and encoded using Base64. By analyzing `cat.jpg` with Aperi'Solve and decoding the suspicious string with CyberChef, we successfully recovered the flag.

4. hashcrack

Category: Cryptography

Challenge Description

A company stored a secret message on a server but was compromised because the administrator used weakly hashed passwords. Our objective is to crack the provided password hashes and retrieve the hidden flag.

Solution

Step 1: Connect to the server

Use `nc` (Netcat) to connect to the challenge server:

```
nc xebec.cylabacademy.net 33291
```

The server displays a welcome message and provides a hash to crack.

Step 2: Crack the MD5 hash

The server provides the first hash:

```
482c811da5d5b4bc6d497ffa98491e38
```

Copy the hash and open [CrackStation](#).

1. Paste the hash into the input field.
2. Complete the CAPTCHA if prompted.
3. Click **Crack Hashes**.
4. Check the result for the plaintext password.

The recovered password is:

hashes: 1 | cracked: 1 | partial: 0 | not found: 0 | invalid: 0

Hash	Type	Result
482c811da5d5b4bc6d497ffa98491e38	md5	password123

Enter it into the Netcat session. The server confirms that the MD5 hash has been cracked.

Step 3: Crack the SHA-1 hash

The server provides the second hash:

```
b7a875fc1ea228b9061041b7cec4bd3c52ab3ce3
```

Repeat the process using CrackStation. The recovered password is:

hashes: 1 | cracked: 1 | partial: 0 | not found: 0 | invalid: 0

Hash	Type	Result
b7a875fc1ea228b9061041b7cec4bd3c52ab3ce3	sha1	letmein

Enter the password into the server to proceed to the next stage.

Step 4: Crack the SHA-256 hash

The server provides the final hash:

```
916e8c4f79b25028c9e467f1eb8eee6d6bbdff965f9928310ad30a8d88697745
```

Paste this hash into CrackStation and inspect the result.

The recovered password is:

hashes: 1 cracked: 1 partial: 0 not found: 0 invalid: 0		
Hash	Type	Result
916e8c4f79b25028c9e467f1eb8eee6d6bbdff965f9928310ad30a8d88697745	sha256	qwerty098

Enter the password into the Netcat session. The server confirms that the SHA-256 hash has been cracked and reveals the flag.

Step 5: Retrieve the flag

The server displays the following flag:

```
academy{UseStr0nG_h@shEs_&PaSswDs!_062c2b77}
```

Flag

```
academy{UseStr0nG_h@shEs_&PaSswDs!_062c2b77}
```

Key Takeaways

- **Hashing:** A one-way process that transforms data into a fixed-length hash value.
- **MD5, SHA-1, and SHA-256:** Different hashing algorithms that produce different hash lengths.
- **CrackStation:** An online tool that compares hashes against precomputed databases to identify their original plaintext values when available.
- **Weak passwords:** Common passwords are easier to recover because attackers can match their hashes against known values.
- **Password security:** Use strong, unique passwords and appropriate password-hashing algorithms such as Argon2id, bcrypt, or scrypt. For

password storage, fast hashes like MD5 and SHA-1 are unsuitable.

Conclusion

This challenge demonstrates how weak passwords can be recovered from their hashes using CrackStation. By cracking the MD5, SHA-1, and SHA-256 hashes in sequence and submitting each recovered password to the server, we obtained the flag.